

# **Introducción a R**

**Maestría en Ciencias Médicas, Odontológicas y de la Salud · UNAM · Semestre  
2027-1**

Dra. Yalbi Itzel Balderas-Martínez      Ing. Luis Alberto Meza Cova

12 de agosto de 2026

# Tabla de contenidos

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Presentación</b>                                | <b>5</b>  |
| Cómo usarlo . . . . .                              | 5         |
| Dónde vamos a trabajar . . . . .                   | 5         |
| Licencia y cita . . . . .                          | 5         |
| <b>1 Encuadre</b>                                  | <b>6</b>  |
| 1.1 Ciencia de datos . . . . .                     | 6         |
| 1.2 Qué es R . . . . .                             | 7         |
| 1.2.1 Ventajas . . . . .                           | 7         |
| 1.2.2 Desventajas . . . . .                        | 7         |
| 1.3 Qué es RStudio . . . . .                       | 8         |
| 1.4 Dónde vamos a trabajar . . . . .               | 8         |
| 1.5 Por qué escribir el análisis . . . . .         | 8         |
| 1.5.1 Un primer ejemplo . . . . .                  | 8         |
| 1.6 De qué trata el curso . . . . .                | 9         |
| 1.7 Evaluación . . . . .                           | 9         |
| 1.8 Material de apoyo . . . . .                    | 9         |
| 1.9 Sesión de R . . . . .                          | 9         |
| <b>2 Primeros pasos en R</b>                       | <b>10</b> |
| 2.1 El caso: dieciocho divisiones a mano . . . . . | 10        |
| 2.2 La consola, el script y el entorno . . . . .   | 10        |
| 2.2.1 La primera línea . . . . .                   | 11        |
| 2.2.2 Cómo se escriben varios comandos . . . . .   | 11        |
| 2.2.3 No vuelvas a escribir lo mismo . . . . .     | 12        |
| 2.3 Pedirle ayuda a R . . . . .                    | 12        |
| 2.3.1 Cómo se lee una página de ayuda . . . . .    | 12        |
| 2.3.2 Ayuda que se ejecuta sola . . . . .          | 13        |
| 2.4 Paquetes . . . . .                             | 13        |
| 2.4.1 Instalar no es cargar . . . . .              | 13        |
| 2.5 Calculadora, asignación y nombres . . . . .    | 14        |
| 2.5.1 Operadores aritméticos . . . . .             | 14        |
| 2.5.2 Asignar: guardar para después . . . . .      | 15        |
| 2.5.3 Cómo nombrar una variable . . . . .          | 15        |
| 2.6 El caso, resuelto . . . . .                    | 16        |
| 2.6.1 Tasa de incidencia . . . . .                 | 16        |
| 2.6.2 Prevalencia y letalidad . . . . .            | 17        |
| 2.7 Demostraciones . . . . .                       | 18        |
| 2.8 Lo que te llevas . . . . .                     | 18        |
| 2.9 Antes de la próxima sesión . . . . .           | 19        |

|              |                                               |           |
|--------------|-----------------------------------------------|-----------|
| <b>3</b>     | <b>Tipos y estructuras de datos</b>           | <b>20</b> |
| 3.1          | El caso: por qué "42" + 10 no es 52           | 20        |
| 3.2          | Los tres tipos básicos                        | 21        |
| 3.2.1        | Los tres errores clásicos                     | 21        |
| 3.2.2        | Un paréntesis: eso que usaste es una función  | 22        |
| 3.3          | Vectores                                      | 22        |
| 3.3.1        | Qué pasa si mezclas tipos                     | 22        |
| 3.3.2        | Otras formas de crear vectores                | 22        |
| 3.3.3        | Nombres                                       | 23        |
| 3.3.4        | Cuatro formas de seleccionar                  | 23        |
| 3.3.5        | Operaciones vectorizadas                      | 24        |
| 3.4          | Matrices                                      | 26        |
| 3.4.1        | R llena las matrices por columnas             | 26        |
| 3.4.2        | Nombres de filas y de columnas                | 27        |
| 3.4.3        | Añadir filas y columnas                       | 27        |
| 3.4.4        | Mirar sin imprimir                            | 27        |
| 3.4.5        | Selección: <code>matriz[fila, columna]</code> | 28        |
| 3.4.6        | Selección por condición                       | 28        |
| 3.4.7        | Combinar condiciones                          | 28        |
| 3.4.8        | Resúmenes por dimensión                       | 29        |
| 3.5          | Factores                                      | 30        |
| 3.5.1        | Contar categorías                             | 30        |
| 3.5.2        | Categorías con orden                          | 31        |
| 3.5.3        | Añadir una categoría                          | 32        |
| 3.5.4        | Para qué sirven de verdad                     | 32        |
| 3.6          | Marcos de datos (data frames)                 | 32        |
| 3.6.1        | Inspeccionarlo                                | 33        |
| 3.6.2        | Seleccionar                                   | 33        |
| 3.6.3        | Filtrar por condición                         | 34        |
| 3.6.4        | Añadir una columna                            | 34        |
| 3.7          | Listas                                        | 35        |
| 3.7.1        | Sacar cosas de una lista                      | 35        |
| 3.7.2        | Por qué importan                              | 36        |
| 3.8          | La escalera, de un vistazo                    | 36        |
| 3.9          | Antes de la próxima sesión                    | 37        |
| <b>I</b>     | <b>Anexos</b>                                 | <b>38</b> |
| <b>Anexo</b> | <b>· Posit Cloud</b>                          | <b>39</b> |
|              | Crear tu cuenta                               | 39        |
|              | Qué vas a ver                                 | 39        |
|              | Por qué en la nube y no en tu computadora     | 39        |
|              | Límite del plan gratuito                      | 40        |
|              | Si prefieres instalarlo en tu computadora     | 40        |
| <b>Anexo</b> | <b>· Glosario</b>                             | <b>41</b> |

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| <b>4 Anexo · Errores frecuentes</b>                                               | <b>42</b> |
| 4.1 Errores que R sí reporta . . . . .                                            | 42        |
| 4.1.1 could not find function "...". . . . .                                      | 42        |
| 4.1.2 objeto 'x' no encontrado . . . . .                                          | 42        |
| 4.1.3 argumento no-numérico para operador binario . . . . .                       | 42        |
| 4.1.4 subíndice fuera de los límites . . . . .                                    | 43        |
| 4.1.5 no se puede abrir la conexión / cannot open file . . . . .                  | 43        |
| 4.2 Avisos que no son errores, y son peores . . . . .                             | 43        |
| 4.2.1 NAs introducidos por coerción . . . . .                                     | 43        |
| 4.2.2 Coerción silenciosa en un vector . . . . .                                  | 44        |
| 4.2.3 longitud del objeto mayor no es múltiplo de la longitud del menor . . . . . | 44        |
| 4.2.4 Un factor al que le asignas un nivel que no existe . . . . .                | 44        |
| 4.3 Resultados que parecen errores y no lo son . . . . .                          | 44        |
| 4.3.1 named numeric(0), character(0), integer(0) . . . . .                        | 44        |
| 4.3.2 NULL . . . . .                                                              | 44        |
| 4.3.3 [1] al principio de cada línea . . . . .                                    | 44        |
| 4.3.4 Una gráfica que no aparece . . . . .                                        | 45        |
| 4.4 Confusiones de concepto . . . . .                                             | 45        |
| 4.5 Cómo pedir ayuda de forma que se pueda contestar . . . . .                    | 45        |
| <b>Referencias</b>                                                                | <b>46</b> |

# Presentación

Este es el manual del curso **Introducción a R** del Programa de Maestría y Doctorado en Ciencias Médicas, Odontológicas y de la Salud de la UNAM, semestre 2027-1.

Las diapositivas son para la clase; este manual es para después: tiene la explicación completa, los ejemplos con su salida y las referencias.

## Cómo usarlo

Cada capítulo corresponde a una sesión. Puedes leerlo en orden o entrar directo al tema que necesitas desde el buscador.

Los bloques de código se pueden copiar con el botón de la esquina y pegar directamente en RStudio.

## Dónde vamos a trabajar

En este curso no instalas nada: usamos **Posit Cloud**, que es RStudio dentro del navegador. Ver el [anexo de Posit Cloud](#) para crear tu cuenta.

## Licencia y cita

El material se puede compartir y adaptar con atribución y sin uso comercial (**CC BY-NC 4.0**). Si lo reutilizas, cita como se indica en el archivo **CITATION.cff** del repositorio.

# 1 Encuadre

Este capítulo es la versión escrita del deck `slides/00-encuadre.qmd`. Su contenido proviene de la clase 0 de la edición anterior del curso, reorganizado para leerse fuera del aula.

## 1.1. Ciencia de datos

La ciencia de datos es un campo interdisciplinario que usa métodos, procesos, algoritmos y sistemas científicos para extraer conocimiento de datos, estén organizados en una tabla o no.

La forma más común de describirla es como la intersección de tres círculos: matemáticas y estadística, cómputo, y **conocimiento del dominio**.

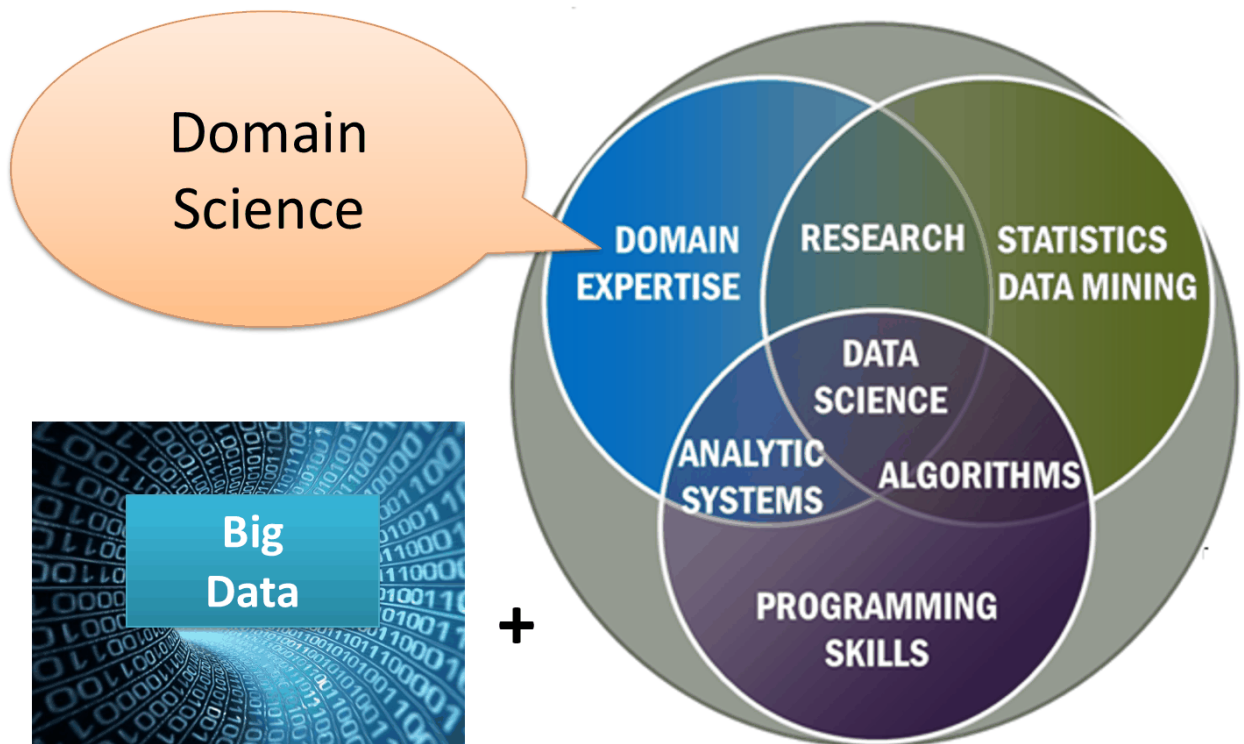


Figura 1.1: Ciencia de datos según el NIST Big Data Working Group

Ese tercer círculo es el que importa aquí. En biología y salud, el conocimiento del dominio es el que tú ya traes: es la parte que no se aprende en un curso de programación.

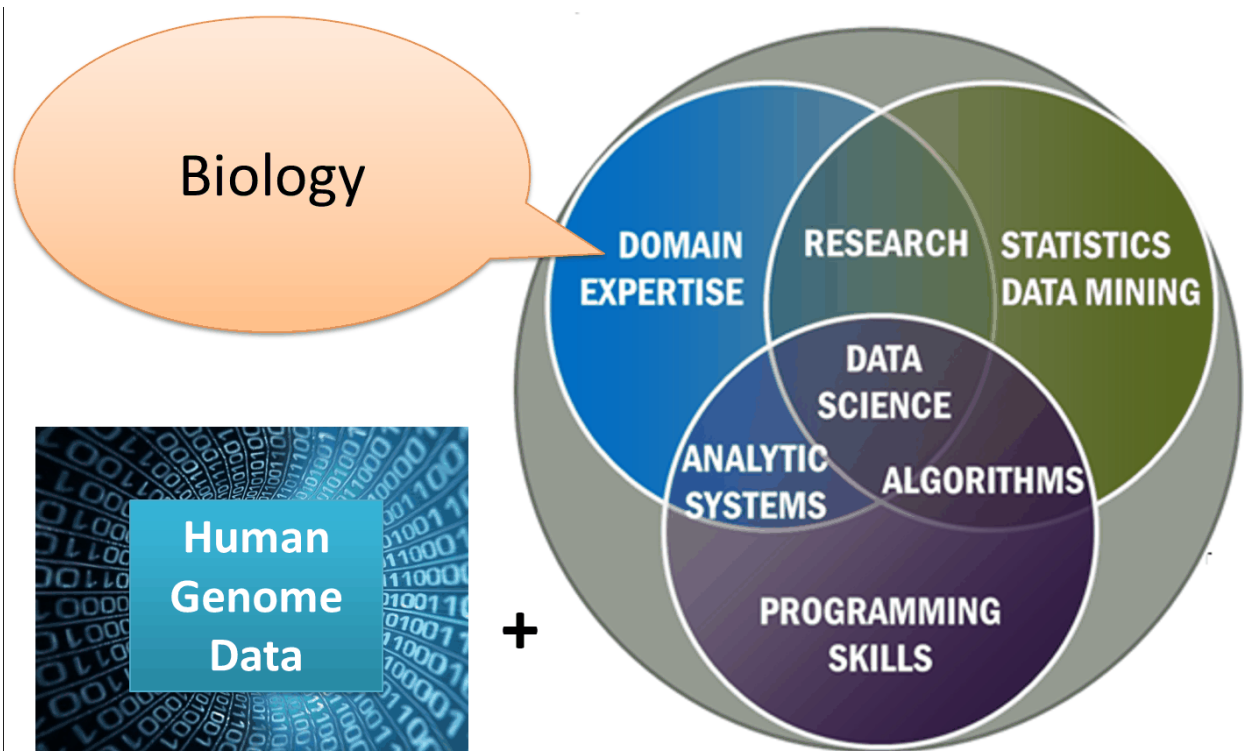


Figura 1.2: El mismo diagrama, aplicado a biología

## 1.2. Qué es R

R es un ambiente de software libre para cómputo estadístico y gráficos. Compila y corre en Windows, macOS y Linux.

Lo desarrollaron personas expertas en estadística a partir del lenguaje S, creado por John Chambers y colegas en los Laboratorios Bell (hoy Lucent Technologies).

### 1.2.1. Ventajas

- Es libre: no depende de una licencia que se venda ni de un presupuesto.
- Las figuras y gráficos que produce son de calidad de publicación.
- Se extiende instalando paquetes.
- Fue hecho por y para gente que analiza datos, no adaptado desde otro fin.

### 1.2.2. Desventajas

- Requiere tiempo de entrenamiento: para personalizar un resultado hay que conocer el lenguaje.
- Algunos paquetes están mal documentados, abandonados, o no son la mejor herramienta para el problema.
- Ciertas tareas se resuelven más fácil en otro lenguaje.

## 1.3. Qué es RStudio

Un ambiente de desarrollo integrado (IDE): consola, editor, herramientas gráficas y manejo del área de trabajo en una sola ventana.

R es el motor; RStudio es el tablero desde el que lo manejas. Puedes usar R sin RStudio, pero es más incómodo.

## 1.4. Dónde vamos a trabajar

En este curso **no instalas nada**. Usamos [Posit Cloud](#): el mismo RStudio, corriendo en un servidor en vez de en tu computadora. Se abre en el navegador con una cuenta gratuita.

La razón es práctica. Instalar R y RStudio en cada computadora convierte la primera sesión en soporte técnico —versiones distintas, permisos de administrador, antivirus— y se va la clase sin haber escrito una línea de R. En la nube todas trabajamos sobre el mismo R, con los mismos paquetes, y un problema se resuelve una sola vez.

No es una versión reducida: lo que aprendas aquí funciona igual el día que instales R en tu máquina.

Cómo crear la cuenta y qué esperar del plan gratuito: ver el [anexo de Posit Cloud](#).

## 1.5. Por qué escribir el análisis

En una hoja de cálculo, las operaciones se hacen a mano y no queda registro de ellas. Seis meses después nadie —ni tú— puede decir con certeza qué celdas se tocaron.

En R, cada paso se escribe como una instrucción, y el archivo de instrucciones **es** el registro del análisis. Si un revisor pregunta cómo obtuviste una cifra, la respuesta es ese archivo: se vuelve a correr y produce el mismo resultado.

Esa propiedad se llama **reproducibilidad** y es la razón de fondo de este curso.

### 1.5.1. Un primer ejemplo

```
edades <- c(34, 41, 28, 55, 47)
mean(edades)
```

```
[1] 41
```

La primera línea guarda cinco edades en un objeto llamado **edades**. La segunda pide su promedio.

El nombre va en plural a propósito: guarda cinco valores, no uno. En la sesión 01 se usa **edad** para un solo número y **edades** para el vector, y conviene que diga lo mismo desde aquí.



## 1.6. De qué trata el curso

Un curso introductorio para aprender a programar en R y adquirir nociones de cómo analizar tus propios datos. No se requieren conocimientos previos de programación.

## 1.7. Evaluación

| Componente          | Peso |
|---------------------|------|
| Tareas en DataCamp  | 20 % |
| Primer examen       | 25 % |
| Segundo examen      | 25 % |
| Proyecto final en R | 30 % |

Asistencia mínima: 80 %.

### **i** Nota

Criterios heredados de la edición anterior del curso. Se confirman para el semestre 2027-1 en [docs/programa-operativo/](#).

## 1.8. Material de apoyo

Además del material de este repositorio, el curso ha usado **DataCamp** a través del programa *DataCamp for the Classroom*, que da acceso completo a la plataforma mientras el curso esté inscrito.

Lecturas de apoyo sobre qué es la ciencia de datos y cómo aprender R:

- <https://www.datacamp.com/es/blog/what-is-data-science-the-definitive-guide>
- <https://www.datacamp.com/blog/learn-r>

## 1.9. Sesión de R

Al cerrar cada análisis conviene registrar con qué versiones se corrió. Es lo que permite explicar, meses después, por qué un resultado cambió:

```
sessioninfo::session_info()
```

## 2 Primeros pasos en R

Tema 01 · unidad teórico-práctica

### Objetivo

Trabajar en R con soltura suficiente para no depender de que alguien dicte el comando: pedirle ayuda al propio R, instalar y cargar un paquete, guardar resultados en variables con nombres que sobrevivan a la semana siguiente, y escribir un cálculo que se pueda volver a correr cuando cambien los datos.

### 2.1. El caso: dieciocho divisiones a mano

Tienes los datos epidemiológicos de seis estados y te piden tres indicadores de cada uno: tasa de incidencia, prevalencia y letalidad. Son tres divisiones por estado. Dieciocho en total.

Con una calculadora se hacen en diez o quince minutos, si no te equivocas. El problema no es ese. El problema llega el mes siguiente, cuando alguien manda la corrección de las cifras de dos estados y hay que rehacerlo todo. Y otra vez en abril.

Ese es el trabajo que R elimina, y por eso el curso abre aquí y no con  $1 + 1$ .

### Lo que hace distinto a R

Una calculadora te da **el resultado**. R te deja **el procedimiento escrito**.

Un procedimiento escrito se vuelve a correr, se revisa, se corrige y se le manda a alguien más.

Un resultado, no.

Es el argumento del curso entero y conviene tenerlo presente en las tres sesiones: no venimos a aprender una herramienta de cálculo, venimos a dejar constancia de cómo se calculó.

### 2.2. La consola, el script y el entorno

Al abrir RStudio aparecen cuatro paneles. Hoy solo importan dos.

| Panel                       | Qué es                                    | Para qué                     |
|-----------------------------|-------------------------------------------|------------------------------|
| <b>Consola</b>              | donde R contesta                          | probar                       |
| <b>Editor</b>               | donde escribes el archivo <code>.R</code> | <b>guardar</b>               |
| Entorno                     | lo que R tiene en memoria                 | ver qué existe               |
| Archivos / Gráficas / Ayuda | el resto                                  | mirar la ayuda y las figuras |

R y RStudio no son lo mismo. **R** es el lenguaje y el intérprete; **RStudio** es la ventana desde la que se le habla. Fuera de este curso se instalan por separado y en ese orden; aquí los dos vienen ya montados en Posit Cloud (ver el [anexo de Posit Cloud](#)).

La diferencia entre los dos primeros paneles es la que ordena todo lo demás: lo que escribes en la consola se ejecuta **y se pierde**; lo que escribes en el script se guarda y se puede volver a correr entero.

### ! La regla de la primera sesión

Se prueba en la consola, se guarda en el script. Todo lo que quieras volver a usar la semana que viene tiene que acabar en un archivo **.R**.

El error de la primera semana casi nunca es de sintaxis: es haber hecho todo el trabajo en la consola y cerrar RStudio. El atajo que evita eso es **Ctrl+Enter** (**Cmd+Enter** en Mac), que manda al intérprete la línea del script donde está el cursor.

## 2.2.1. La primera línea

```
3 + 4
#> [1] 7
```

El [1] no es parte del resultado: es **la posición del primer elemento** de lo que R está imprimiendo. Con un solo número parece ruido. Va a tener sentido completo en el tema de tipos y estructuras de datos, cuando el resultado tenga cien elementos y R los reparta en varias líneas: cada línea empieza indicando por qué elemento va.

## 2.2.2. Cómo se escriben varios comandos

```
# Con salto de línea: lo normal
casos <- 125
poblacion <- 5000

# Con punto y coma: dos en la misma línea
casos <- 125; poblacion <- 5000

# Entre llaves: un grupo que se ejecuta junto
{ casos <- 125; poblacion <- 5000 }
```

El **#** comenta hasta el final de la línea. El punto y coma existe y se lee peor; sirve para dos asignaciones cortas y emparentadas, no para juntar tres cosas distintas.

Hay un límite que parece curiosidad y explica un error real: **un comando introducido en la consola no puede pasar de 4 095 bytes**. Si pegas un bloque enorme copiado de internet, se corta. En un script no pasa.

### 2.2.3. No vuelvas a escribir lo mismo

- $\uparrow$  y  $\downarrow$  en la consola recorren lo que ya escribiste.
- **Ctrl**+ $\uparrow$  (**Cmd**+ $\uparrow$ ) abre la lista completa del historial.
- **Tab** completa nombres de funciones, de variables y de rutas de archivo.

De los tres, el más rentable es **Tab**. Completa y, de paso, te dice si el objeto existe: si no aparece en la lista, o lo escribiste mal o todavía no lo has creado.

## 2.3. Pedirle ayuda a R

R trae su documentación dentro. No hace falta salir a buscar en internet para saber qué argumentos acepta una función.

| Escribes                               | Cuándo                         | Qué te da                                        |
|----------------------------------------|--------------------------------|--------------------------------------------------|
| <code>?read.table</code>               | <b>sabes</b> el nombre exacto  | la página de ayuda de esa función                |
| <code>??solve</code>                   | no lo sabes exacto             | busca en todo lo instalado                       |
| <code>help.search("data input")</code> | buscas por tema                | lo mismo que <code>??</code> , admitiendo frases |
| <code>apropos("lm")</code>             | recuerdas un trozo del nombre  | los nombres que <b>contienen</b> ese trozo       |
| <code>find("lowess")</code>            | sabes el nombre, no el paquete | en qué paquete vive                              |

La regla corta: **una interrogación si sabes el nombre, dos si no**.

`help.start()` abre la documentación completa en el navegador. Es útil el primer día y casi nunca después.

### 2.3.1. Cómo se lee una página de ayuda

Todas tienen las mismas secciones, en el mismo orden:

| Sección            | Qué contesta                                            |
|--------------------|---------------------------------------------------------|
| <b>Description</b> | qué hace                                                |
| <b>Usage</b>       | cómo se llama, con sus argumentos y valores por defecto |
| <b>Arguments</b>   | qué significa cada argumento                            |
| <b>Value</b>       | <b>qué devuelve</b>                                     |
| <b>Examples</b>    | código que puedes copiar y correr                       |

Casi nadie las lee de arriba abajo. Lo práctico es ir a **Usage**, saltar a **Examples**, y volver a **Arguments** cuando algo no cuadra.

Aprender a leer la ayuda vale más que memorizar veinte funciones: las veinte funciones se buscan, y leer la ayuda no se aprende solo.

### 2.3.2. Ayuda que se ejecuta sola

```
example(sum)
#> sum> sum(1:5)
#> [1] 15
#> sum> sum(1:5, NA)
#> [1] NA
#> sum> sum(1:5, NA, na.rm = TRUE)
#> [1] 15
```

`example()` corre los ejemplos de la página de ayuda dentro de tu sesión. Este en concreto es un regalo: en tres líneas enseña la función, enseña que un valor faltante (`NA`) contamina el resultado, y enseña el argumento que lo excluye.

De momento basta con esta idea de `NA`: hay un hueco en los datos, y R prefiere avisarte a inventarse un número. Vuelve en el tema de tipos y estructuras de datos.

💡 Ejercicio · encuentra la función

1. ¿En qué paquete vive `lowess`?
2. Busca funciones cuyo nombre contenga `mean`. ¿Cuántas salen?
3. Abre la ayuda de `round`. ¿Qué hace el argumento `digits`?
4. Corre los ejemplos de `round` sin copiarlos a mano.

Respuestas: `find("lowess")` · `apropos("mean")` · `?round` · `example(round)`.

La segunda no tiene un número correcto: depende de los paquetes que tengas instalados. Esa es la lección.

## 2.4. Paquetes

Cuando instalas R ya vienen paquetes de base: `base`, `stats`, `utils`, `graphics` y algunos más. `sum()`, `c()` y `mean()` viven ahí. El índice completo del paquete `base` está en la documentación oficial (R Core Team, 2024).

Lo demás lo escribe gente ajena al equipo que desarrolla R y se publica en **CRAN**, el repositorio oficial. Que R sea extensible por terceros es la razón de que exista `ggplot2`, que es todo el tema 04.

### 2.4.1. Instalar no es cargar

```
install.packages("ggplot2") # UNA vez, en la vida de tu computadora
library(ggplot2)           # CADA vez que abres R
```

`install.packages()` **descarga** el paquete a tu disco. `library()` lo **pone disponible** en la sesión que tienes abierta. Son dos cosas distintas y hacen falta las dos.

⚠ El error de la primera semana

```
Error in ggplot(...) : could not find function "ggplot"
```

El paquete está instalado. Lo que falta es `library(ggplot2)` **en esta sesión**.

La analogía que funciona: instalar es comprar el libro; cargar es sacarlo del estante y abrirlo. Comprarlos dos veces no sirve de nada, y tenerlo en el estante tampoco si no lo abres.

Por eso, en los scripts que se comparten, la línea de `install.packages()` va **comentada**: si no, se reinstala el paquete cada vez que alguien corre el archivo.

```
# install.packages("ggplot2") # comentado a propósito
library(ggplot2)
d <- data.frame(x = rnorm(100))
ggplot(data = d) + geom_histogram(aes(x = x), bins = 30)
```

Esas cuatro líneas generan cien números al azar de una distribución normal, los meten en una tabla y dibujan su histograma. Hoy no hay que entenderlas: hay que ver que **funcionan**. El resto del curso consiste en entenderlas —la tabla en el tema de tipos y estructuras de datos, y la gráfica en el de ggplot2—.

## 2.5. Calculadora, asignación y nombres

### 2.5.1. Operadores aritméticos

```
3 + 4      #> 7
3 - 4      #> -1
3 * 4      #> 12
3 / 4      #> 0.75
3 ^ 4      #> 81
7 %/% 2    #> 3      cociente entero
7 %% 2     #> 1      resto
```

Los dos últimos parecen exóticos y no lo son: `%/%` es como se pregunta en R si un número es par (`x %% 2 == 0`).

💡 Ejercicio · piensa un número

Piensa un número cualquiera. Multiplícalo por 5, súmale 1, multiplica el resultado por 2, réstale 12, divide entre 10 y réstale el número inicial.

A todo el mundo le da **−1**.

No es magia, es álgebra:

$$\frac{(5x + 1) \cdot 2 - 12}{10} - x = \frac{10x - 10}{10} - x = x - 1 - x = -1$$

El número que pensaste se cancela. Escribirlo en R con una variable — `x <- 7; ((x * 5 + 1) * 2 - 12) / 10 - x` — y probar con varios valores es el argumento más corto a favor de escribir el procedimiento y no solo el resultado.

### 2.5.2. Asignar: guardar para después

Asignar es guardar un valor en la memoria del ordenador e identificarlo con un nombre, para acceder a él rápidamente después.

```
x <- 4 # el operador de R
x = 5 # también funciona
```

Los dos operadores asignan. **En R se usa <-**, y no es capricho: el signo = ya significa otra cosa dentro de una llamada a función.

```
seq(from = 0, to = 1, by = 0.25)
```

Ahí `from = 0` **no crea una variable**: pasa un argumento con nombre. El operador <- significa siempre lo mismo, en cualquier posición, así que quien lea tu código sabrá sin pensar cuál de las dos cosas está pasando.

El atajo de RStudio para escribirlo es `Alt+-` (`Option+-` en Mac). Conviene aprenderlo el primer día: si hay que teclear los dos caracteres a mano, se acaba usando =.

Y hay algo que sorprende al principio: **asignar no imprime nada**. Para ver el contenido de un objeto se escribe su nombre.

```
tasa_incendencia <- 25 # no imprime nada
tasa_incendencia      # ahora sí
#> [1] 25
```

Quien venga de otro lenguaje va a buscar un `print()`. Existe, y aquí no hace falta.

### 2.5.3. Cómo nombrar una variable

R distingue mayúsculas de minúsculas: `casos`, `Casos` y `CASOS` son tres variables distintas. Y aun no es aún.

| Regla                                                   | Mal                  | Bien                     |
|---------------------------------------------------------|----------------------|--------------------------|
| No empezar con dígito                                   | <code>1casos</code>  | <code>casos1</code>      |
| El <code>_</code> no va al principio                    | <code>_casos</code>  | <code>casos_marzo</code> |
| Tras un <code>.</code> inicial debe ir letra, no dígito | <code>.1casos</code> | <code>.casos</code>      |

| Regla                          | Mal       | Bien           |
|--------------------------------|-----------|----------------|
| Máximo 256 bytes               | —         | nombres cortos |
| Portable: solo A-Z a-z 0-9 _ . | población | poblacion      |

Los acentos y las ñes funcionan en muchos sistemas y fallan en otros. Lo peor es que fallan al abrir el archivo en otra computadora, es decir, lejos de donde se cometió el error. **Regla de la casa: sin acentos ni ñes en los nombres de objeto**; en los comentarios, los que hagan falta.

Más allá de lo que R acepta, está lo que se entiende:

```
# Se entiende hoy y dentro de un mes
casos_confirmados <- 125
poblacion_riesgo <- 5000
tasa_incidencia <- (casos_confirmados / poblacion_riesgo) * 1000

# Se entiende hoy
a <- 125; b <- 5000
c <- (a / b) * 1000
```

El segundo bloque no está mal: está mudo. Y tiene un problema añadido —`c` pisa el nombre de la función `c()`, que es la que más se usa al trabajar con vectores—. Pisar nombres de funciones básicas produce errores que aparecen mucho después y son difíciles de rastrear.

La convención de este curso, que es la del material entero: minúsculas y guion bajo (`snake_case`), sin acentos, con un sustantivo que diga qué guarda. Nada de `camelCase`.

## 2.6. El caso, resuelto

### 2.6.1. Tasa de incidencia

Mide la aparición de **casos nuevos** de una enfermedad en una población en riesgo, durante un periodo de tiempo.

$$TI = \frac{C}{N} \times k$$

donde  $C$  son los casos nuevos en el periodo,  $N$  la población en riesgo y  $k$  una constante de escala (1 000; 100 000...). La constante solo cambia la unidad en la que se lee el resultado —«por cada mil habitantes», «por cada cien mil»—; no cambia el fenómeno.

```
casos_confirmados <- 125      # casos nuevos
poblacion_riesgo <- 5000     # población en riesgo

# Tasa de incidencia por cada 1 000 habitantes
tasa_incidencia <- (casos_confirmados / poblacion_riesgo) * 1000
```



```
tasa_incidencia
#> [1] 25
```

Cada elemento de la fórmula tiene una línea de código con su nombre. Los paréntesis no hacen falta —la división y la multiplicación tienen la misma precedencia y se evalúan de izquierda a derecha— y están para que el código se lea como la fórmula.

## 2.6.2. Prevalencia y letalidad

$$P = \frac{C}{N} \times 100 \quad L = \frac{D}{C} \times 100$$

La **prevalencia** mide la proporción de personas que tienen la enfermedad en un momento dado: aquí  $C$  son **todos** los casos, nuevos y antiguos. La **letalidad** mide la proporción de personas con la enfermedad que mueren a causa de ella: su denominador son **los casos**, no la población.

### ⚠ La confusión clásica

Cambiar el denominador de la letalidad —poner la población en lugar de los casos— cambia el resultado por un factor de veinticinco en el ejemplo de abajo. Y no lanza ningún mensaje: R divide lo que le pongas.

```
casos_confirmados <- 125      # casos nuevos
poblacion_riesgo  <- 5000     # personas en riesgo
muertes           <- 8        # defunciones entre los casos
casos_totales     <- 200      # todos los casos de la enfermedad

tasa_incidencia <- (casos_confirmados / poblacion_riesgo) * 1000
prevalencia     <- (casos_totales     / poblacion_riesgo) * 100
tasa_letalidad  <- (muertes           / casos_totales)    * 100

tasa_incidencia
#> [1] 25
prevalencia
#> [1] 4
tasa_letalidad
#> [1] 4
```

`prevalencia` y `tasa_letalidad` valen 4 las dos y no significan lo mismo: el 4% de la población tiene la enfermedad, y el 4% de los enfermos murió. La coincidencia es de este ejemplo, no del mundo, y es un buen recordatorio de que el número solo no dice nada: hace falta saber qué se dividió entre qué. Por eso las variables se llaman como se llaman.

### 💡 Ejercicio 1 · cambia los valores

Asigna otros números de casos confirmados y de población en riesgo, y vuelve a calcular la tasa de incidencia.

**Pregunta:** ¿qué pasa si la población en riesgo crece y los casos confirmados se quedan igual? Fíjate en lo que costó: cambiar **dos líneas** y volver a correr. Eso es exactamente lo que la calculadora no permitía, y es la promesa con la que abrió la sesión.

### 💡 Ejercicio 2 · dos poblaciones

Asigna valores para dos poblaciones distintas —`poblacion_riesgo1` y `poblacion_riesgo2`—, calcula la tasa de incidencia en las dos y compáralas.

**Pregunta:** ¿cuál de las dos poblaciones tiene mayor riesgo relativo?

Con dos poblaciones ya hay cuatro variables. Con los seis estados del caso serían doce, y el código dejaría de caber en la pantalla. Esa incomodidad tiene nombre —se llama **vector**— y ocupa una línea. Es el tema de tipos y estructuras de datos.

## 2.7. Demostraciones

R trae demostraciones que enseñan de lo que es capaz sin que haya que escribir nada:

```
demo(persp)      # superficies en perspectiva
demo(graphics)   # el repertorio de gráficos base
demo(Hershey)     # tipografías vectoriales
demo(plotmath)   # fórmulas matemáticas dentro de una gráfica
```

No entran en el examen de nada. Sirven para ver el techo de la herramienta el primer día, que es cuando más falta hace.

## 2.8. Lo que te llevas

- Se prueba en la consola, **se guarda en el script**.
- Una interrogación si sabes el nombre; dos si no.
- `install.packages()` una vez; `library()` cada sesión.
- `<-` para asignar, siempre.
- Nombres en minúsculas, con `_`, sin acentos, y que digan qué guardan.

Y si algo se te olvida, que no sea esto: **el valor de R no es que calcule, es que deja escrito cómo calculó.**

## 2.9. Antes de la próxima sesión

1. Entra a tu proyecto de Posit Cloud y comprueba que abre sin errores (ver el [anexo de Posit Cloud](#)). No tienes que instalar nada.
2. Corre `install.packages("ggplot2")` una vez en tu proyecto: se usa en la tema 04.
3. Termina los ejercicios 1 y 2 en un archivo `.R` guardado.

Si algo no instala, escíbeme antes del miércoles. Resolver una instalación rota en vivo cuesta quince minutos de clase; por correo, cinco.

## 3 Tipos y estructuras de datos

Tema 02 · unidad teórico-práctica

### Objetivo

Elegir la estructura de datos que corresponde al problema —vector, matriz, factor, data frame o lista— y justificar la elección; y seleccionar subconjuntos de datos por posición, por nombre y por condición lógica sin escribir un solo ciclo.

Este capítulo es una escalera, no una lista de temas. Cada estructura aparece porque **la anterior se queda corta** con el problema que sigue. Si se lee como cinco cosas que memorizar, se pierde lo único que importa: reconocer qué te está pidiendo el problema que tienes delante.

### 3.1. El caso: por qué "42" + 10 no es 52

```
y <- 10
num <- "42"
y + num
#> Error in y + num : argumento no-numérico para operador binario
```

Para ti los dos «son 42 y 10». Para R, uno es un **número** y el otro es **texto**. El mensaje de error es de los buenos: dice exactamente qué pasó.

Es, además, el error que más aparece en la vida real, cuando se abre un CSV en el que una columna traía un espacio de más, un guion o una coma decimal.

```
class(num)
#> [1] "character"
class(y)
#> [1] "numeric"

num <- as.numeric(num) # convertir el texto a número
y + num
#> [1] 52
```

`class()` es la función que más se usa hoy, y `as.numeric()` inaugura una familia entera: `as.character()`, `as.logical()`, `as.factor()`.

### ⚠ Advertencia

`as.numeric("hola")` devuelve **NA con una advertencia**, no con un error. Convertir a ciegas es como se pierden datos en silencio: conviene mirar cuántos NA aparecieron después de convertir.

## 3.2. Los tres tipos básicos

```
a <- 5          # numérico
b <- 3.14       # numérico
c2 <- 2e3       # numérico: 2000

x <- 5; y <- 10
x < y          # lógico: TRUE
x == y         # lógico: FALSE

nombre <- "Luis"    # carácter
curso  <- 'IntroR'  # las comillas simples son equivalentes
```

Tres apuntes sobre esto:

- `a <- 5` da "numeric", no "integer". R guarda los números como `double` por defecto; para un entero de verdad hace falta escribir `5L`.
- Las comillas dobles y las simples son equivalentes. Elige unas y no las mezcles dentro de un archivo.
- El nombre `c2` está a propósito: `c` pisaría la función `c()`, que es la que se usa en todo el resto del capítulo (Sección 2.5.3).

### 3.2.1. Los tres errores clásicos

```
"42" + 10      # 1. operar entre tipos incompatibles -> Error

logico <- TRUE  # 2. "logical"
texto  <- "TRUE" # "character" - no es lo mismo

nombre <- Luis  # 3. sin comillas -> Error: objeto 'Luis' no encontrado
```

El tercero es el traicionero. Sin comillas, R no interpreta `Luis` como texto: busca **un objeto** llamado `Luis`. Si no existe, avisa. Si existe, obedece y te da su contenido, que casi nunca es lo que querías. Un error se convierte así en un bug silencioso.

### 3.2.2. Un paréntesis: eso que usaste es una función

`class()` y `as.numeric()` son **funciones**: bloques de código con nombre que cumplen una tarea y se reutilizan tantas veces como haga falta, en vez de repetir líneas. Se reconocen por los paréntesis que siguen al nombre, y `?nombre` dice qué argumentos aceptan.

Escribir funciones propias es materia de otro curso. Pero conviene nombrar la idea aquí, porque de este punto en adelante todo lo que se hace es llamar funciones.

## 3.3. Vectores

Un **vector** es una colección **ordenada** de elementos **del mismo tipo**.

```
v_num <- c(1, 2, 3, 4.5)
v_int <- c(1L, 2L, 3L)      # la L fuerza un entero de verdad
v_log <- c(TRUE, FALSE, TRUE)
v_chr <- c("a", "b", "c")
```

`c()` viene de *combine*. Y «del mismo tipo» no es un detalle de implementación: es la definición.

### 3.3.1. Qué pasa si mezclas tipos

```
c(1, 2, "tres")
#> [1] "1" "2" "tres"

c(TRUE, FALSE, 3)
#> [1] 1 0 3
```

R **no da error**: convierte todo al tipo que lo admite todo. Se llama **coerción** y sigue este orden: lógico → numérico → carácter.

#### Advertencia

Tus números se volvieron texto **sin avisar**. Tres líneas más abajo, un `sum()` falla, y el error aparece muy lejos de donde se originó.

Regla práctica: si un `sum()` o una comparación fallan sin motivo aparente, `class()` al vector antes que nada.

Esta es la razón profunda de que a R le importen los tipos, y la que explica por qué el capítulo empieza por ahí y no por las estructuras.

### 3.3.2. Otras formas de crear vectores

```
1:6
#> [1] 1 2 3 4 5 6

seq(from = 0, to = 1, by = 0.25)
#> [1] 0.00 0.25 0.50 0.75 1.00

rep(0, times = 5)
#> [1] 0 0 0 0 0
```

1:6 es el atajo de cada día. `seq()` hace falta cuando el paso no es 1, y `rep()` cuando hay que inicializar algo.

### 3.3.3. Nombres

Imagina una lista de casos confirmados en distintos estados. No basta con tener los números: hace falta saber a qué estado corresponde cada uno.

```
casos_marzo <- c(1200, 850, 430, 670, 1500, 920)
names(casos_marzo) <- c("CDMX", "Jalisco", "Yucatan",
                        "Puebla", "NuevoLeon", "Chiapas")

casos_marzo
#>      CDMX      Jalisco      Yucatan      Puebla NuevoLeon      Chiapas
#>      1200       850       430       670       1500       920
```

#### ! Importante

`names()` **no junta dos vectores**. Le pega una etiqueta a cada posición del vector que ya existe: los datos siguen siendo exactamente los mismos.  
`names` es un **atributo** del vector, no una segunda columna. Sirve para identificar cada posición con un texto, y para leer sin contar posiciones: `casos_marzo["Puebla"]` devuelve 670.

Aquí se cierra lo que quedó pendiente en el Capítulo 2: aquellas doce variables sueltas para seis estados son ahora dos vectores.

### 3.3.4. Cuatro formas de seleccionar

```
defunciones_marzo <- c(50, 40, 12, 25, 65, 30)

defunciones_marzo[1]           #> 50           por posición
defunciones_marzo[c(2, 5)]     #> 40 65         varias
defunciones_marzo[-3]          #> 50 40 25 65 30   quita la tercera
defunciones_marzo[defunciones_marzo > 30] #> 50 40 65   por condición
```

Dos cosas sorprenden aquí:

- Los índices empiezan en 1, no en 0. Quien venga de Python lo va a tropezar durante un par de sesiones.
- El índice negativo excluye, no cuenta desde el final. [-3] significa «todos menos el tercero».

Y la cuarta forma es la importante, porque se repite en todas las estructuras que quedan.

```
defunciones_marzo > 30
#>      CDMX      Jalisco      Yucatan      Puebla NuevoLeon      Chiapas
#>      TRUE       TRUE      FALSE      FALSE       TRUE       FALSE
```

La condición **no devuelve los datos**: devuelve un vector de TRUE y FALSE del mismo largo. Ese vector es lo que va dentro de los corchetes, y R se queda con las posiciones que valen TRUE.

Entender ese paso intermedio es lo que hace que las matrices y los data frames sean fáciles en lugar de mágicos. Vale la pena correrlo en dos pasos: primero la condición sola, después dentro de los corchetes.

### 3.3.5. Operaciones vectorizadas

```
casos_enero <- c(3000, 1500, 1200, 1800, 2500, 2100)
names(casos_enero) <- names(casos_marzo)

casos_enero + casos_marzo
#>      CDMX      Jalisco      Yucatan      Puebla NuevoLeon      Chiapas
#>      4200      2350      1630      2470      4000      3020
```

R aplica la operación **elemento por elemento**, posición por posición, sin que escribas un ciclo. Se llama **vectorización** y es la razón de que R resulte cómodo para trabajar con datos.

Quien haya programado en otro lenguaje espera un **for**. En R casi nunca hace falta, y cuando aparece suele ser señal de que existe una forma más corta.

Junto a esa familia hay otra, la de **agregación**, que resume el vector entero en un solo número:

```
sum(casos_enero)    #> 12100
sum(casos_marzo)    #> 5570
mean(casos_enero)   #> 2016.667
min(casos_marzo)    #> 430
max(casos_marzo)    #> 1500
```

| Familia                                  | Devuelve                         |
|------------------------------------------|----------------------------------|
| <b>Elemento a elemento</b> (+, *, >)     | un vector <b>del mismo largo</b> |
| <b>Aggregación</b> (sum, mean, min, max) | <b>un solo número</b>            |

Tener clara esa distinción es lo que hace que `colSums()` y `rowSums()`, más abajo, no necesiten explicación.



### 💡 Ejercicio 3 · letalidad de enero

La letalidad se define como  $L = \frac{D}{C} \times 100$ , donde  $D$  son las defunciones y  $C$  el total de casos.  
¿Qué estados tuvieron una letalidad mayor a 3 en el mes de enero?

```
casos_enero      <- c(3000, 1500, 1200, 1800, 2500, 2100)
defunciones_enero <- c(100, 60, 25, 30, 80, 55)
names(casos_enero) <- names(defunciones_enero) <- names(casos_marzo)
```

**Respuesta.** Dos líneas: una calcula, otra filtra.

```
letalidad <- (defunciones_enero / casos_enero) * 100
letalidad
#>      CDMX    Jalisco    Yucatan    Puebla NuevoLeon    Chiapas
#> 3.333333 4.000000 2.083333 1.666667 3.200000 2.619048

letalidad[letalidad > 3]
#>      CDMX    Jalisco NuevoLeon
#> 3.333333 4.000000 3.200000
```

Fíjate en que los nombres **viajan con los datos** a través de la división y del filtro. No hubo que volver a decir qué estado era cada número: por eso valía la pena ponerlos.

### 💡 Ejercicio 4 · enero contra marzo

¿En qué estados la letalidad fue mayor en enero que en marzo?

**Respuesta.**

```
letalidad_marzo <- (defunciones_marzo / casos_marzo) * 100

letalidad > letalidad_marzo
#>      CDMX    Jalisco    Yucatan    Puebla NuevoLeon    Chiapas
#>    FALSE    FALSE    FALSE    FALSE    FALSE    FALSE

letalidad[letalidad > letalidad_marzo]
#> named numeric(0)
```

### ❗ Importante

`named numeric(0)` **no es un error**. Dice cuatro cosas: es un vector, es numérico, tiene nombres y su longitud es cero. En ningún estado se cumplió la condición, y esa es la respuesta correcta.

Un resultado vacío asusta la primera vez. Verlo a propósito, en un caso donde «ninguno» es la respuesta buena, evita leerlo como fallo la próxima vez.

## 3.4. Matrices

Una **matriz** es una estructura bidimensional —filas y columnas, como una tabla— en la que **todos los elementos son del mismo tipo**.

Aparece cuando el vector se queda corto. Por ejemplo, con la composición nutricional de cuatro alimentos por cada 100 g:

|                 | kcal | Carbohidratos | Proteínas | Grasas |
|-----------------|------|---------------|-----------|--------|
| Manzana         | 52   | 14            | 0.3       | 0.2    |
| Pollo (pechuga) | 165  | 0             | 31        | 3.6    |
| Arroz blanco    | 130  | 28            | 2.7       | 0.3    |
| Frijoles        | 127  | 23            | 9         | 0.5    |

Se puede hacer con cuatro vectores. Pero entonces la relación entre ellos —que la primera posición de los cuatro es la manzana— deja de estar en los datos y pasa a estar en tu cabeza.

### 3.4.1. R llena las matrices por columnas

```
matrix(1:9, nrow = 3, ncol = 3)
#>      [,1] [,2] [,3]
#> [1,]    1    4    7
#> [2,]    2    5    8
#> [3,]    3    6    9
```

#### ⚠ Advertencia

El 1, 2, 3 quedó en la **primera columna**, no en la primera fila. Es la sorpresa clásica, y si no se detecta aquí se arrastra hasta la matriz de nutrientes, donde ya no es evidente porque los números no están ordenados.

Prueba `matrix(1:12, nrow = 3, ncol = 4)` y busca dónde cae el 4.

Cuando cada vector es **una fila** —un alimento— hay que decirlo:

```
manzana <- c(52, 14, 0.3, 0.2); pollo <- c(165, 0, 31, 3.6)
arroz <- c(130, 28, 2.7, 0.3); frijol <- c(127, 23, 9, 0.5)

nutricion_matrix <- matrix(c(manzana, pollo, arroz, frijol),
                           nrow = 4, byrow = TRUE)
```

Sin `byrow = TRUE` la matriz sale transpuesta y con el sentido físico equivocado: la primera columna sería «manzana» y la primera fila «las calorías de los cuatro».

### 3.4.2. Nombres de filas y de columnas

```
colnames(nutricion_matrix) <- c("Calorias", "Carbohidratos",  
                                "Proteinas", "Grasas")  
rownames(nutricion_matrix) <- c("Manzana", "Pollo", "Arroz", "Frijoles")  
nutricion_matrix  
#>           Calorias Carbohidratos Proteinas Grasas  
#> Manzana           52           14         0.3  0.2  
#> Pollo             165            0        31.0  3.6  
#> Arroz             130           28         2.7  0.3  
#> Frijoles          127           23         9.0  0.5
```

En vectores era `names()`. En matrices **no**: hay dos dimensiones y hay que especificar cuál. Intentar `names(matriz) <- ...` no falla con un mensaje claro; simplemente no hace lo que esperabas.

### 3.4.3. Añadir filas y columnas

```
huevo    <- c(155, 1.1, 13, 11)  
lentejas <- c(116, 20, 9, 0.4)  
nutricion_extendida <- rbind(nutricion_matrix, rbind(huevo, lentejas))  
  
fibra <- c(2.4, 0, 0.1, 7.0, 0, 8.0)  
sodio <- c(1, 74, 1, 2, 124, 2)  
nutricion_final <- cbind(nutricion_extendida, Fibra = fibra, Sodio_mg = sodio)
```

`rbind()` pega **filas** (*row bind*) y `cbind()` pega **columnas** (*column bind*). Las columnas nuevas llevan **seis** valores, uno por cada fila que ya tiene la matriz; si el largo no coincide, R recicla en silencio o falla, según el caso.

Un detalle que se ve en la salida: `rbind(huevo, lentejas)` toma los nombres de las variables como nombres de fila, así que `huevo` y `lentejas` salen en minúscula y desentonan con los cuatro de arriba. Es un buen momento para hablar de consistencia en los nombres.

### 3.4.4. Mirar sin imprimir

```
class(nutricion_matrix)  
#> [1] "matrix" "array"  
  
str(nutricion_matrix)  
#>  num [1:4, 1:4] 52 165 130 127 14 0 28 23 0.3 31 ...  
#>  - attr(*, "dimnames")=List of 2  
#>   ..$ : chr [1:4] "Manzana" "Pollo" "Arroz" "Frijoles"  
#>   ..$ : chr [1:4] "Calorias" "Carbohidratos" "Proteinas" "Grasas"
```

`str()` da una vista compacta de la **estructura**: dimensiones, tipo y una muestra del contenido. `summary()` da un resumen **estadístico** —mínimo, cuartiles, mediana, media, máximo— y solo funciona si la matriz es numérica.

`str()` es la función que más se usa en la vida real y la que menos se enseña. Con una matriz de 4×4 parece innecesaria; con una de veinte mil filas es lo único que se puede hacer.

#### ! Importante

Regla de la casa: **nunca imprimas un objeto del que no sabes el tamaño**. Primero `str()` o `dim()`.

### 3.4.5. Selección: `matriz[fila, columna]`

```
nutricion_final["Manzana", "Calorias"]    #> 52
nutricion_final["Manzana", ]              # toda la fila
nutricion_final[, "Calorias"]              # toda la columna
nutricion_final[1, 1]                     # lo mismo, por posición
nutricion_final[c(2:3), ]                 # varias filas
nutricion_final[, c("Proteinas", "Grasas")]
```

Dejar el hueco vacío significa «todas», y la coma nunca se omite: es lo que separa filas de columnas. Olvidarla —`nutricion_final["Manzana"]`— no da un error claro, da un resultado raro.

### 3.4.6. Selección por condición

Funciona igual que en los vectores, en la posición de las filas:

```
high_cal <- nutricion_final[, "Calorias"] > 100
high_cal
#>  Manzana  Pollo  Arroz Frijoles  huevo lentejas
#>   FALSE   TRUE   TRUE   TRUE   TRUE   TRUE

nutricion_final[high_cal, ]    # se queda con las cinco filas TRUE
```

### 3.4.7. Combinar condiciones

Para responder preguntas con más de un criterio se combinan vectores lógicos con tres operadores:

| Operador | Devuelve TRUE si...          |
|----------|------------------------------|
| & (Y)    | las dos condiciones son TRUE |
| \  (O)   | al menos una lo es           |
| ! (NO)   | invierte: TRUE pasa a FALSE  |

---

---

|          |                     |
|----------|---------------------|
| Operador | Devuelve TRUE si... |
|----------|---------------------|

---

---

¿Qué alimentos tienen más de 10 g de proteína Y menos de 15 g de carbohidratos?

```
condicion_A <- nutricion_final[, "Proteinas"] > 10
condicion_B <- nutricion_final[, "Carbohidratos"] < 15

nutricion_final[condicion_A & condicion_B, ]
#>      Calorias Carbohidratos Proteinas Grasas Fibra Sodio_mg
#> Pollo      165           0.0        31   3.6    0       74
#> huevo      155           1.1        13  11.0    0      124
```

Guardar cada condición en su propia variable no es adorno: permite imprimirlas por separado cuando el resultado no es el esperado. Es la técnica de depuración de este bloque, y sin ella una selección compuesta que devuelve algo raro no se puede diagnosticar.

¿Qué alimentos tienen más de 5 g de fibra O menos de 1 g de grasa?

```
nutricion_final[nutricion_final[, "Fibra"] > 5 |
                 nutricion_final[, "Grasas"] < 1, ]
#>  Manzana, Arroz, Frijoles, lentejas
```

¿Cuáles tienen poco sodio y poca grasa, pero no son la manzana?

```
low_sod <- nutricion_final[, "Sodio_mg"] < 10
low_gras <- nutricion_final[, "Grasas"] < 10
es_manzana <- rownames(nutricion_final) == "Manzana"

nutricion_final[low_sod & low_gras & !es_manzana, ]
#>  Arroz, Frijoles, lentejas
```

`rownames()` devuelve un vector de texto; compararlo con `==` produce el vector lógico que `!` invierte.

### 3.4.8. Resúmenes por dimensión

```
colSums(nutricion_matrix)
#>      Calorias Carbohidratos      Proteinas      Grasas
#>      474.0         65.0         43.0         4.6

colMeans(nutricion_matrix)
#>      Calorias Carbohidratos      Proteinas      Grasas
#>      118.50         16.25         10.75         1.15
```

```
macros <- nutricion_final[, c("Carbohidratos", "Proteinas", "Grasas")]
rowSums(macros)
#> Manzana      Pollo      Arroz Frijoles      huevo lentejas
#>      14.5      34.6      31.0      32.5      25.1      29.4
```

### ⚠ Advertencia

La suma de la columna **Calorías** existe como número y no significa nada: son kcal de alimentos distintos. `rowSums(macros)` **sí** significa algo —gramos de macronutrientes por alimento— porque las tres columnas comparten unidad. R calcula lo que le pidas. No valida el sentido de la operación; eso te toca a ti.

## 3.5. Factores

¿Qué pasa cuando una variable solo puede tomar un número limitado de valores? Por ejemplo, clasificar los alimentos en «Fruta», «Proteína» o «Carbohidrato».

Para eso están los **factores**: vectores pensados para datos **categoricos**.

```
alimentos <- c("Fruta", "Proteína", "Carbohidrato",
               "Proteína", "Carbohidrato", "Fruta")
factor_alimentos <- factor(alimentos)
factor_alimentos
#> [1] Fruta      Proteína    Carbohidrato Proteína    Carbohidrato Fruta
#> Levels: Carbohidrato Fruta Proteína
```

Los **niveles** (*levels*) son las categorías que R encontró. Por dentro, R no guarda seis textos: guarda seis enteros y una tabla que dice que el 1 es «Carbohidrato», el 2 «Fruta» y el 3 «Proteína». Si la categoría se repite mil veces, la diferencia de memoria y de velocidad es grande.

Pero la eficiencia no es la razón principal por la que existen; eso viene al final del apartado.

### 3.5.1. Contar categorías

```
summary(factor_alimentos)
#> Carbohidrato      Fruta      Proteína
#>           2           2           2

table(factor_alimentos)
#> factor_alimentos
#> Carbohidrato      Fruta      Proteína
#>           2           2           2
```

### ! Importante

`summary()` de una matriz numérica daba cuartiles. De un factor da un **conteo**. La misma función se comporta según el tipo del objeto que le des.

Ese es un rasgo grande del diseño de R, y es la razón por la que `class()` importa tanto.

### 3.5.2. Categorías con orden

Algunas variables categóricas tienen un orden intrínseco: «Bajo», «Medio» y «Alto» no son intercambiables.

```
porciones <- c("Mediana", "Pequeña", "Grande", "Mediana", "Pequeña")

factor_porciones <- factor(porciones,
                           levels = c("Pequeña", "Mediana", "Grande"),
                           ordered = TRUE)

factor_porciones
#> [1] Mediana Pequeña Grande Mediana Pequeña
#> Levels: Pequeña < Mediana < Grande

factor_porciones[1] > factor_porciones[2] # Mediana > Pequeña
#> [1] TRUE
```

### ⚠ El orden alfabético es una trampa

Si no escribes `levels`, R los ordena **alfabéticamente**: Grande < Mediana < Pequeña, que es justo al revés.

Lo peligroso es que nada avisa: el factor queda ordenado, las comparaciones funcionan y el resultado está invertido. Sobrevive hasta la gráfica final, donde las categorías salen al revés y nadie sabe por qué.

**Si el factor es ordenado, `levels` se escribe siempre a mano.**

### 💡 Ejercicio 5 · opiniones

```
evaluacion_sabor <- c("Bueno", "Excelente", "Regular",
                     "Bueno", "Excelente")
```

Conviértelo en un factor **ordenado** llamado `factor_sabor`, con los niveles en el orden lógico Regular < Bueno < Excelente, y cuenta cuántas opiniones hay de cada tipo con `summary()`.

**Respuesta.**

```
factor_sabor <- factor(evaluacion_sabor,
                      levels = c("Regular", "Bueno", "Excelente"),
                      ordered = TRUE)

summary(factor_sabor)
```

```
#> Regular      Bueno Excelente
#>      1          2          2
```

Comprueba que el orden del conteo es el de `levels` y no el alfabético: ahí se ve que escribirlo sirvió de algo.

### 3.5.3. Añadir una categoría

Un factor no acepta valores que no estén en sus niveles. Primero se declara el nivel; después se asigna el valor.

```
dietas <- c("Omnívora", "Vegetariana", "Vegana", "Omnívora", "Vegetariana")
factor_dieta <- factor(dietas)

levels(factor_dieta) <- c(levels(factor_dieta), "Keto") # primero el nivel
factor_dieta[6] <- "Keto"                             # después el valor
```

Si te saltas el primer paso, R pone NA **con una advertencia**, no con un error.

### 3.5.4. Para qué sirven de verdad

- `lm()`, `glm()` y compañía generan **automáticamente** el análisis por grupo cuando una variable es factor.
- `ggplot2` le da **un color, una faceta o una caja** a cada nivel, como se ve en el capítulo de visualización con `ggplot2`.
- El orden de los niveles decide el orden en que salen las categorías en cualquier tabla o gráfica.

Que una variable sea factor o texto **cambia el resultado**. No es un tecnicismo.

## 3.6. Marcos de datos (data frames)

¿Y cuando cada columna es de un tipo distinto? El nombre de un alimento es texto, sus calorías son un número y su grupo nutricional es un factor. Una matriz no puede: exige un solo tipo.

Para eso está el **data frame**, que es:

- **una lista de vectores**: cada columna es un vector;
- de **tipos distintos**: numérico, carácter, lógico, factor, todos juntos;
- del **mismo largo**: todas las columnas con el mismo número de filas.



```

nutricion_df <- data.frame(
  Nombre      = c("Manzana", "Pollo", "Arroz",
                  "Frijoles", "Huevo", "Lentejas"),
  Calorias    = c(52, 165, 130, 127, 155, 116),
  Grupo       = factor(c("Fruta", "Proteína", "Carbohidrato",
                        "Legumbre", "Proteína", "Legumbre")),
  Vegetariano = c(TRUE, FALSE, TRUE, TRUE, FALSE, TRUE)
)

```

Esta es la estructura de casi todo conjunto de datos real: cualquier CSV o archivo de Excel que cargues se convierte en un data frame.

### 3.6.1. Inspeccionarlo

```

str(nutricion_df)
#> 'data.frame':    6 obs. of  4 variables:
#> $ Nombre      : chr  "Manzana" "Pollo" "Arroz" "Frijoles" ...
#> $ Calorias    : num  52 165 130 127 155 116
#> $ Grupo       : Factor w/ 4 levels "Carbohidrato",...: 2 4 1 3 4 3
#> $ Vegetariano: logi  TRUE FALSE TRUE TRUE FALSE TRUE

```

Los enteros 2 4 1 3 4 3 de la línea de **Grupo** son los niveles codificados: «Fruta» es el 2 porque los niveles van en orden alfabético. Es la prueba visible de lo que se explicó en el apartado de factores.

| Función                                                        | Para qué                                       |
|----------------------------------------------------------------|------------------------------------------------|
| <code>str()</code>                                             | estructura y tipos — <b>la primera siempre</b> |
| <code>summary()</code>                                         | resumen estadístico por columna                |
| <code>head()</code> · <code>tail()</code>                      | las 6 primeras o últimas filas                 |
| <code>dim()</code> · <code>nrow()</code> · <code>ncol()</code> | tamaño                                         |
| <code>names()</code> · <code>colnames()</code>                 | nombres de columna                             |

`head(nutricion_df, n = 3)` muestra solo las tres primeras.

### 3.6.2. Seleccionar

```

nutricion_df$Calorias      #> [1]  52 165 130 127 155 116
nutricion_df[["Nombre"]]   # útil si el nombre está guardado en una variable
nutricion_df[, 2]          # por posición

nutricion_df[1, ]          # primera fila, todas las columnas
nutricion_df[c(1, 2), c("Nombre", "Grupo")] # filas y columnas

```

**\$** es el que se usa el noventa por ciento de las veces. Y con **Tab** después del **\$**, RStudio ofrece los nombres de columna: es uno de los trucos que más cambian la experiencia del primer mes.

### 3.6.3. Filtrar por condición

```
nutricion_df[nutricion_df$Calorias > 120, ]
#>   Nombre Calorias      Grupo Vegetariano
#> 2   Pollo     165   Proteína      FALSE
#> 3   Arroz     130 Carbohidrato      TRUE
#> 4 Frijoles    127   Legumbre      TRUE
#> 5   Huevo     155   Proteína      FALSE

nutricion_df[nutricion_df$Vegetariano == TRUE &
  nutricion_df$Calorias < 120, ]
#>   Nombre Calorias      Grupo Vegetariano
#> 1 Manzana      52    Fruta      TRUE
#> 6 Lentejas    116 Legumbre      TRUE
```

Es el **mismo mecanismo** que en vectores y en matrices: una condición que produce TRUE/FALSE, metida entre corchetes. Tercera vez que aparece en este capítulo, y no son tres reglas: es una.

Los números de fila que sobreviven —2, 3, 4, 5 y luego 1, 6— son los del data frame original, no una reenumeración.

### 3.6.4. Añadir una columna

```
nutricion_df$Proteinas <- c(0.3, 31, 2.7, 9, 13, 9)
```

Eso es todo: asignarle un vector con \$. El vector tiene que tener exactamente tantos elementos como filas. Si tiene menos y su largo es divisor del número de filas, R lo **recicla** sin avisar; si no lo es, falla. El aviso práctico es «cuenta tus filas».

#### Ejercicio 6 · un data frame propio

Crea `bebidas_df` con tres bebidas y estas columnas:

| Columna    | Valores                                                 | Tipo          |
|------------|---------------------------------------------------------|---------------|
| Nombre     | Agua · Jugo de Naranja ·<br>Refresco de Cola            | carácter      |
| Volumen_ml | 500 · 250 · 355                                         | numérico      |
| Tipo       | Bebida sin azúcar · Jugo de<br>fruta · Bebida azucarada | <b>factor</b> |

Después:

1. verifica la estructura con `str()`;
2. muestra solo la columna `Volumen_ml`;
3. filtra las bebidas de más de 300 ml;
4. añade una columna lógica `Requiere_Refrigeracion`, TRUE solo para el jugo.

La respuesta completa está en `notebooks/R/02-estructuras-de-datos.qmd`.

## 3.7. Listas

Una **lista** almacena una colección de objetos de **cualquier tipo y cualquier longitud**: vectores, data frames, matrices, incluso otras listas. Sirve para encapsular datos heterogéneos.

Un data frame exige que todas las columnas midan lo mismo. Una lista, no.

```
datos_casos <- data.frame(
  ID_Caso      = c("C01", "C02", "C03"),
  Edad         = c(25, 42, 31),
  Hospitalizado = c(FALSE, TRUE, FALSE)
)
matriz_contactos <- matrix(c(0, 1, 1, 1, 0, 0, 1, 0, 0), nrow = 3)

investigacion_brote <- list(
  Nombre      = "Nuevo brote",      # texto de largo 1
  Datos       = datos_casos,        # un data frame de 3 x 3
  Contactos   = matriz_contactos,   # una matriz de 3 x 3
  Activo      = TRUE                # un lógico de largo 1
)

str(investigacion_brote)
#> List of 4
#> $ Nombre : chr "Nuevo brote"
#> $ Datos  : 'data.frame': 3 obs. of 3 variables:
#> $ Contactos: num [1:3, 1:3] 0 1 1 1 0 0 1 0 0
#> $ Activo : logi TRUE
```

### 3.7.1. Sacar cosas de una lista

Hay tres operadores y la diferencia entre ellos es la que más tiempo hace perder al empezar:

| Operador                 | Qué devuelve                                                  |
|--------------------------|---------------------------------------------------------------|
| <code>\$Datos</code>     | el <b>objeto</b> que hay dentro                               |
| <code>[["Datos"]]</code> | el <b>objeto</b> , y además admite índice numérico            |
| <code>["Datos"]</code>   | una <b>sublista</b> — su clase sigue siendo <code>list</code> |

| Operador | Qué devuelve |
|----------|--------------|
|----------|--------------|

```
class(investigacion_brote[["Nombre"]]) #> [1] "character"
class(investigacion_brote$Nombre)      #> [1] "list"
```

### ! Importante

Un corchete devuelve **una caja más chica**. Dos corchetes devuelven **lo que hay dentro de la caja**.

Añadir o modificar elementos funciona con los mismos operadores, y se pueden anidar:

```
investigacion_brote[["Ubicacion"]] <- "Hospital Central"
investigacion_brote$Datos$Edad <- c(25, 42, 31)
```

### 3.7.2. Por qué importan

Porque son **el formato de salida estándar de muchas funciones estadísticas de R**. Cuando corras `lm()` en otro curso, lo que devuelve es una lista. Saber extraerle elementos es lo que permite usar el resultado en lugar de solo mirarlo.

## 3.8. La escalera, de un vistazo

| Estructura        | Guarda                       | Se rompe cuando...                  |
|-------------------|------------------------------|-------------------------------------|
| <b>Vector</b>     | un tipo, una dimensión       | necesitas filas <b>y</b> columnas   |
| <b>Matriz</b>     | un tipo, dos dimensiones     | las columnas son de tipos distintos |
| <b>Factor</b>     | categorías, con o sin orden  | — es un vector especializado        |
| <b>Data frame</b> | tipos distintos, mismo largo | los elementos no miden lo mismo     |
| <b>Lista</b>      | cualquier cosa               | — no se rompe, por eso es la última |

No hay que memorizar cinco estructuras. Hay que reconocer **qué te está pidiendo el problema**.

Y sobre todo, lo que se repitió tres veces:

```
v[v > 30]           # vector
m[m[, "Calorias"] > 100, ] # matriz
df[df$Calorias > 120, ] # data frame
```

Una sola idea —la condición devuelve un vector lógico, ese vector va entre corchetes, R se queda con lo que valga **TRUE**— en tres sintaxis. La misma reaparece en el capítulo de visualización con `ggplot2` para decidir qué se dibuja.

### 3.9. Antes de la próxima sesión

1. Termina el ejercicio 6 en un archivo `.R` guardado.
2. Comprueba que `library(ggplot2)` funciona **sin error**.
3. Corre `str(mpg)` y mira qué tipo tiene cada columna.

El punto 3 no es de relleno: `mpg` es el conjunto de datos con el que se trabaja todo el tema 04, y lo primero que haremos es preguntarnos qué columna se puede mapear a qué.

# **Parte I**

## **Anexos**

## Anexo · Posit Cloud

En este curso **no instalas nada**. Trabajamos en **Posit Cloud**, que es RStudio corriendo dentro del navegador.

### Crear tu cuenta

1. Entra a <https://posit.cloud>.
2. Elige **Sign Up** y el plan **Cloud Free**.
3. Puedes registrarte con tu correo, o con tu cuenta de Google o de GitHub.
4. Confirma el correo de verificación.

Con eso ya tienes RStudio. No hay descarga ni instalación.

### Qué vas a ver

La misma ventana de RStudio que usarías en tu computadora:

- **Consola**, abajo a la izquierda: donde escribes y R responde.
- **Editor**, arriba a la izquierda: donde guardas tu código.
- **Environment**, arriba a la derecha: los objetos que has creado.
- **Files / Plots / Packages**, abajo a la derecha.

No es una versión reducida ni un simulador: es RStudio. Lo que aprendas aquí funciona igual el día que instales R en tu máquina.

### Por qué en la nube y no en tu computadora

Instalar R y RStudio en cada computadora convierte la primera clase en soporte técnico: versiones distintas, permisos de administrador, rutas con acentos, antivirus. Se va la sesión y nadie aprendió R.

En la nube trabajamos sobre el mismo R, con los mismos paquetes. Si algo falla, falla igual para todo el grupo y se resuelve una vez.

## Límite del plan gratuito

El plan **Cloud Free** incluye un número limitado de horas de cómputo al mes —alrededor de 15— y un tope de proyectos.

Para las sesiones del curso alcanza, pero conviene saber dos cosas:

- **Cierra el proyecto cuando termines.** Un proyecto abierto sigue consumiendo horas aunque no estés escribiendo. Es la causa más común de quedarse sin horas a media semana.
- Puedes revisar tu consumo en la sección de tu cuenta.

### Nota

Las condiciones del plan gratuito las fija Posit y cambian de vez en cuando. Verifica el límite vigente en <https://posit.cloud/plans>.

## Si prefieres instalarlo en tu computadora

Se puede, y al final del curso lo vemos con calma. El orden importa: primero R, después RStudio, porque RStudio necesita que R ya exista para encontrarlo.

- R — <https://cran.r-project.org>
- RStudio Desktop — <https://posit.co/download/rstudio-desktop/>

No es requisito del curso. Si decides hacerlo, hazlo **además** de Posit Cloud, no en lugar de: las sesiones se dan sobre el entorno de la nube.



## Anexo · Glosario

Términos técnicos que aparecen en el curso, definidos en el punto donde se usan por primera vez y recogidos aquí para consulta rápida.

**Objeto.** Un nombre al que le asignas un valor para poder reutilizarlo.

**Función.** Una instrucción que recibe algo, hace una operación y devuelve un resultado.

**Vector.** Una secuencia de valores del mismo tipo.

**Data frame.** Una tabla: columnas con nombre, cada una del mismo largo.

**Reproducibilidad.** Que otra persona, con el mismo código y los mismos datos, obtenga el mismo resultado.

**Repositorio.** La carpeta del proyecto con su historial de cambios versionado.

## 4 Anexo · Errores frecuentes

Este anexo está ordenado por **lo que ves en la consola**, no por tema. Cuando algo falla, busca aquí el mensaje.

Casi todos estos errores aparecen a lo largo del curso y ninguno es señal de que lo estés haciendo mal: son la forma en que R avisa, y aprender a leerlos es parte del curso.

### 4.1. Errores que R sí reporta

#### 4.1.1. could not find function "..."

```
Error in ggplot(...) : could not find function "ggplot"
```

Falta `library(ggplot2)` en esta sesión. Instalar y cargar son dos cosas distintas (Sección 2.4.1).

También puede ser una errata: R distingue mayúsculas de minúsculas, así que `Ggplot()` no existe. **Tab** completa el nombre y de paso confirma que existe.

#### 4.1.2. objeto 'x' no encontrado

```
Error: objeto 'casos' no encontrado
```

Tres causas, en orden de frecuencia:

1. **Todavía no lo creaste.** Suele pasar al correr un script desde la mitad.
2. **Lo escribiste distinto.** `casos` no es `Casos`, y aun no es aún.
3. **Se te olvidaron las comillas** alrededor de un texto: `nombre <- Luis` hace que R busque un objeto llamado `Luis`.

#### 4.1.3. argumento no-numérico para operador binario

```
Error in y + num : argumento no-numérico para operador binario
```

Estás sumando un número con un texto. Comprueba con `class()` y convierte con `as.numeric()`. Se ve en el tema de tipos y estructuras de datos.

En datos reales, esto casi siempre significa que una columna que debía ser numérica se leyó como texto porque traía un espacio, un guion o una coma decimal.

#### 4.1.4. subíndice fuera de los límites

Error in `x[[10]]` : subíndice fuera de los límites

Pediste el elemento 10 de algo que tiene menos. Comprueba el tamaño con `length()`, `dim()` o `str()` antes de indexar.

Recuerda que **en R los índices empiezan en 1**, no en 0.

#### 4.1.5. no se puede abrir la conexión / cannot open file

R no encuentra el archivo. Comprueba dónde está buscando:

```
getwd()      # directorio de trabajo actual
list.files()  # qué hay ahí
```

La solución duradera no es `setwd()`: es trabajar dentro de un **proyecto de RStudio** (`.Rproj`), que fija el directorio de trabajo en la carpeta del proyecto.

## 4.2. Avisos que no son errores, y son peores

Un error para la ejecución. Un aviso la deja seguir con datos equivocados.

#### 4.2.1. NAs introducidos por coerción

```
as.numeric("hola")
#> [1] NA
#> Warning message: NAs introducidos por coerción
```

Convertiste a número algo que no lo era, y ese valor ahora es NA. Si conviertes una columna entera, cuenta cuántos NA aparecieron:

```
sum(is.na(mi_columna))
```

Si el número no es cero, ve a mirar esas filas antes de seguir.

### 4.2.2. Coerción silenciosa en un vector

```
c(1, 2, "tres")
#> [1] "1"      "2"      "tres"
```

**Sin ningún aviso.** Tus números se volvieron texto y el error va a aparecer mucho más abajo, cuando un `sum()` falle.

Regla: si una operación numérica falla sin motivo aparente, `class()` al vector antes que nada.

### 4.2.3. longitud del objeto mayor no es múltiplo de la longitud del menor

R estaba **reciclando** un vector corto para completar uno largo, y no le salió exacto. El reciclado en sí no avisa cuando el largo es divisor, así que este mensaje es la punta del problema.

Cuenta tus filas antes de asignar una columna nueva.

### 4.2.4. Un factor al que le asignas un nivel que no existe

```
factor_dieta[6] <- "Keto"
#> Warning message: invalid factor level, NA generated
```

Primero se declara el nivel, después se asigna el valor. Los factores se ven en el tema de tipos y estructuras de datos.

## 4.3. Resultados que parecen errores y no lo son

### 4.3.1. `named numeric(0)`, `character(0)`, `integer(0)`

Un vector **vacío**. Ninguna posición cumplió la condición. Es una respuesta válida y a veces es la respuesta correcta.

### 4.3.2. NULL

El elemento no existe. En una lista, `mi_lista$Inexistente` devuelve `NULL` en vez de dar error, lo cual es cómodo y también una fuente de confusión.

### 4.3.3. `[1]` al principio de cada línea

No es parte del resultado: es la **posición del primer elemento** que R está imprimiendo en esa línea. Con vectores largos verás `[1]`, `[18]`, `[35]`...

#### 4.3.4. Una gráfica que no aparece

Dentro de un script o una función, `ggplot()` construye el objeto pero no lo dibuja si no lo imprimes. En la consola se dibuja solo; dentro de un `for` o de una función, hay que envolverlo en `print()`.

### 4.4. Confusiones de concepto

| Síntoma                                                   | Causa real                                                                |
|-----------------------------------------------------------|---------------------------------------------------------------------------|
| «Instalé el paquete y sigue sin funcionar»                | falta <code>library()</code> en esta sesión                               |
| «Le puse <code>names()</code> a la matriz y no pasó nada» | en matrices son <code>rownames()</code> y <code>colnames()</code>         |
| «Los números salieron en la columna equivocada»           | <code>matrix()</code> llena por columnas; falta <code>byrow = TRUE</code> |
| «Las categorías salen al revés en la gráfica»             | factor ordenado sin <code>levels</code> escrito a mano                    |

«`geom_point(aes(colour = "blue"))` no pinta de azul» | dentro de `aes()` van variables, fuera van valores |

«Mis barras cuentan en vez de usar mi total» | `geom_bar()` cuenta; querías `geom_col()` |

«Perdí todo el trabajo al cerrar RStudio» | estaba en la consola, no en un script guardado |

### 4.5. Cómo pedir ayuda de forma que se pueda contestar

Cuando escribas —al canal del curso o por correo— incluye estas cuatro cosas:

1. **Qué querías hacer**, en una frase.
2. **El código exacto** que corriste, copiado y pegado, no descrito.
3. **El mensaje completo**, copiado y pegado, no una captura recortada.
4. La salida de `sessionInfo()`, que dice tu versión de R y qué paquetes tienes cargados.

Con eso, casi cualquier problema se resuelve en una respuesta. Sin eso, hacen falta tres.

# Referencias

R Core Team. (2024). *An Introduction to R*. R Foundation for Statistical Computing. <https://cran.r-project.org/doc/manuals/r-release/R-intro.html>